Track Malfunctions in Flatland

Baruch Shteken Lena Friedek

Railway Scheduling, Universität Potsdam
https://github.com/wug-on-praat/TRailMix

21st Sep 2026

Malfunctions in Flatland

- solve.py handles train malfunctions.
- Base encoding for scheduling and forwarding information; secondary encoding for rescheduling (Gilkra [2] and Murphy [3]).
- Information forwarded:
  - actions as constraints;

```
:- not action(train(0),move_forward,22).
:- not action(train(1),move_forward,22).
:- not action(train(1),wait,24).
```

  - the malfunction atom;

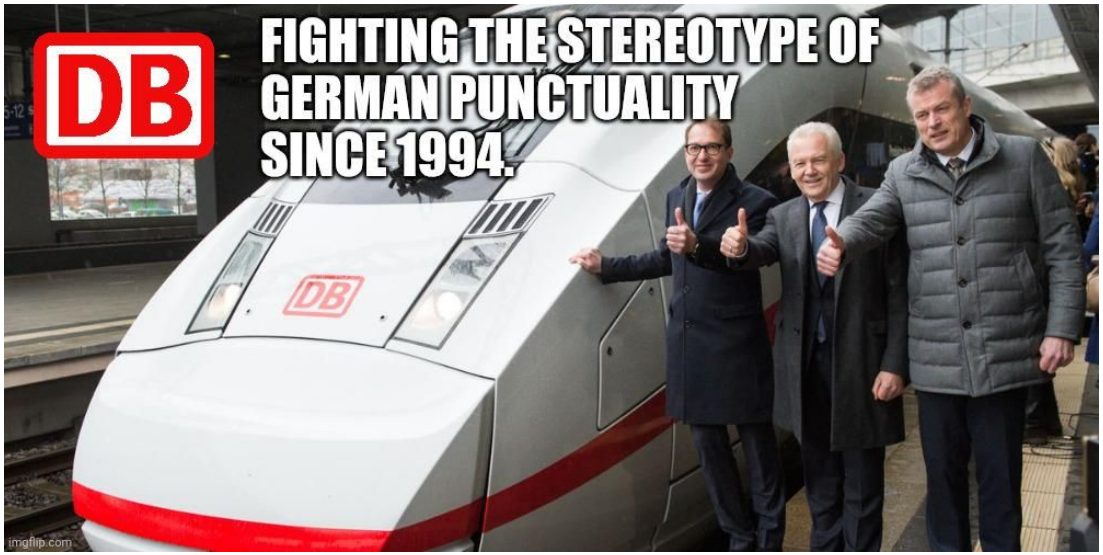
```
malfunction(1,6,23).
```

  - planned_action.

```
planned_action(train(0),move_forward,0).
planned_action(train(0),move_forward,1).
```

Track Malfunctions

- Real-world issue
- Negatively affects punctuality
- Reasons: weather, fires, trespassing, repairs, high-frequent use



(Source)

Our Contribution

TRailMix [1]

- Create malfunction (randomly/deterministically)
- Forwarding additional information
- Malfunction logging
- Visualization in Flatland

(Source)

Create a Track Malfunction

```
def create_trkmalf(self, timestep, predefined_mals):

  if not predefined_mals:
    malfunction_ind = random.random()

    if malfunction_ind > 0.99:
      malfunction_cell = tuple(random.choice(self.grid))
      malfunction_duration = random.randint(2,20)
      mal_timestep = timestep

    else:
      return([])
  ...
  self.track_malfunctions.append((malfunction_cell,
    malfunction_duration))
  self.track_record.append((malfunction_cell, mal_timestep,
    malfunction_duration))
  return([(malfunction_cell, malfunction_duration)])

```

Additional Forwarded Information

- track malfunction

```
track_malfunction((17, 19), 21, 13).
```

- position constraint rules

```
:- position( _, (17, 19), 14, _).
:- position( _, (17, 19), 15, _).
```

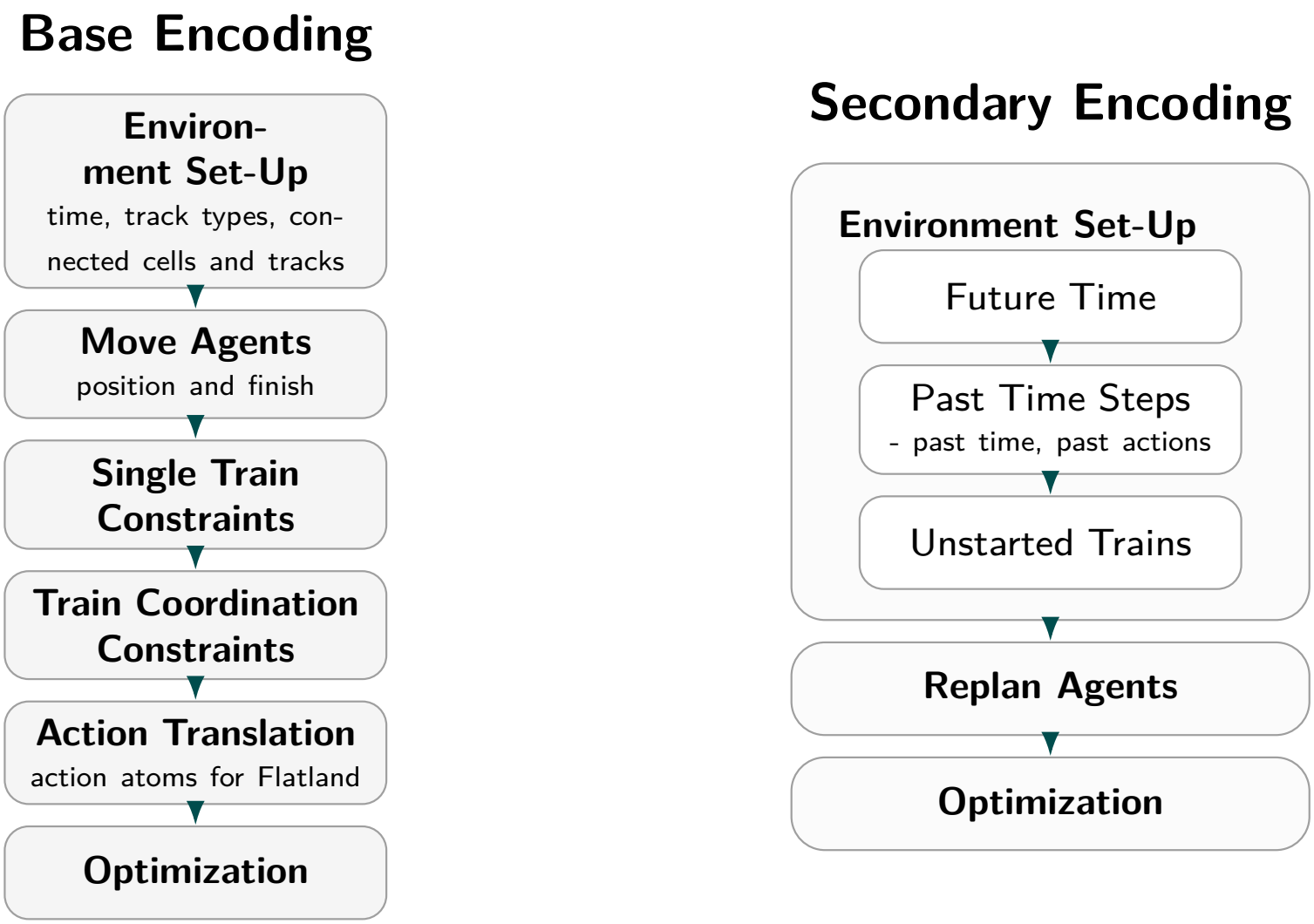
- planned position

```
planned_position(0, (20, 3), 15, s).
planned_position(1, (28, 25), 15, n).
```

- current position

```
current_position(0, (19, 3), 14, s).
current_position(1, (29, 25), 14, n).
```

Base and Secondary Encoding Overview



Rescheduling

The basis of our secondary encoding is to determine which trains are affected by the malfunction and reschedule only them unless there is a collision and then reschedule also the trains affected by the collision

- Check if there is a consequence

```
consequence(train(ID), P, T) :- planned_position(ID, P, T, _),
track_malfunction(P, Dur_mal, T_mal), Dur_mal + T_mal >= T.
```

- Use as many planned trains as possible

```
{planned(ID)}:- train(ID), not consequence(train(ID), _, _).
#maximize { 1@3, ID : planned(ID)}.
```

- Recalculate schedule for unplanned trains

```
calculate(ID) :- consequence(train(ID), _, _).
calculate(ID) :- train(ID), not planned(ID).
```

Experiment 1 - Heuristic vs. no Heuristic

- Does the Heuristic improve solving?
- Prefer planned trains:

```
#heuristic planned(ID): train(ID). [100, true]
```

- **Results - Yes!**
  - Small environments: no difference
  - Big/dense environments: heuristic significantly faster (especially in no consequence scenarios - 4 mins)

  → Less re-proving necessary

Experiment 2 - Base vs. Secondary

- Is Secondary really better than Redo Planning (Waldow [4])?

- **Results - Yes!**
  - Small environments: little/ no difference
  - Big/dense environments: secondary encoding significantly faster (2 mins in no consequence)

  $\rightarrow$ successful use of forwarded information

Prioritization

The encodings are different only in their optimization rule prioritization

- First Plan Prioritization

```
#maximize { 1@3, ID : planned(ID)}.
#minimize { 1@2, ID, T : action(train(ID), _, T) }.
```

- Time Prioritization

```
#maximize { 1@2, ID : planned(ID)}.
#minimize { 1@3, ID, T : action(train(ID), _, T) }.
```

Experiment 3 - Base vs. Time Prio

- Is Secondary with Time Prioritization better than Redo Planning?

- **Results - Yes??**
  - No consequence: base is faster (40 s in large, dense environment) or similar
  - Other consequences: secondary is significantly faster (50% in a complex scenario)

  → Time Prio struggles with direct translation of original plan

Experiment 4 - First Plan vs. Time Prioritization

- Is Time Prioritization better than First Plan Prioritization?

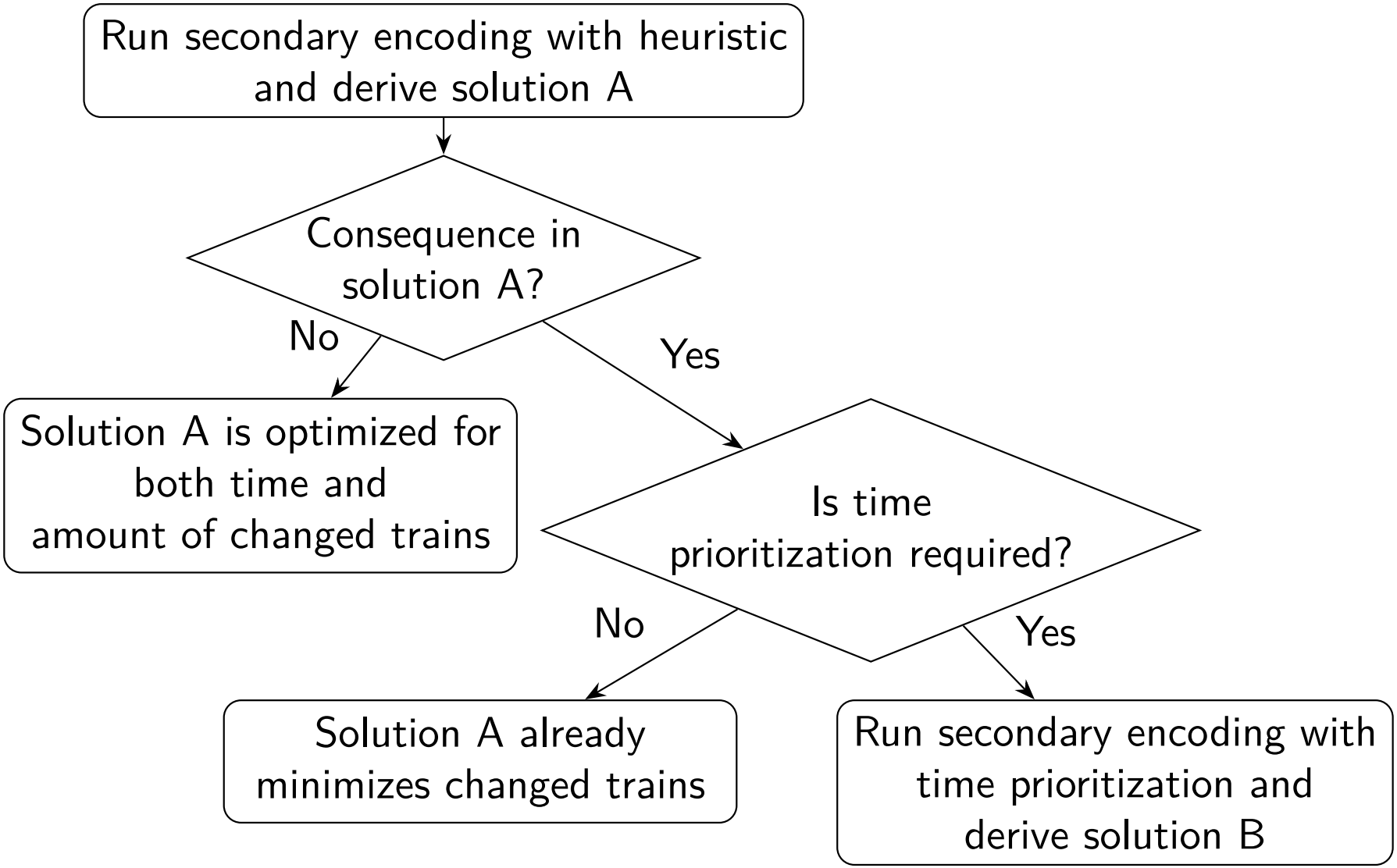
- **Results - No!**
  - First plan encoding is significantly faster in almost all environments and scenarios
  - Similar performance in small environments

  $\rightarrow$ First Plan Prio is faster BUT...

Differences in Output Quality

Figure: Qualitative differences between encodings.

Discussion


Run secondary encoding with heuristic and derive solution A
Consequence in solution A?
No
Yes
Solution A is optimized for both time and amount of changed trains
Is time prioritization required?
No
Yes
Solution A already minimizes changed trains
Run secondary encoding with time prioritization and derive solution B

Future Work

- Add different random approaches to track malfunctions, e.g frequented tracks might be more prone to a malfunction
- Seperate scheduling and rescheduling processes in order to run rescheduling without the scheduling part
- Enhance to code to accept the creation of multiple track malfunction per timestep
- Visualize the scheduling answer if no rescheduling answer is possible

References

- Friedek, L., Shteken, B.: Trailmix. https://github.com/wug-on-praat/TRailMix (2026), gitHub repository
- Gili i Bueno, G., Kratzsch, F.: Flatland: Malfunctions research on train training (2025), https://github.com/krr-up/railway-research/blob/main/railway-course/2024-wise/gilkra2025a.pdf
- Murphy, K.R.: Flatland (2026), https://github.com/krr-up/flatland, Accessed: 17.08.2026
- Waldow, I.: Intelligent malfunction handling in large railway systems (2025), https://github.com/krr-up/railway-research/blob/main/theses/waldow2025a.pdf